Benchmarking Modern Document Intelligence: A Comparative Study of OCR, Layout-Aware Parsers, and Vision-Language Models

DocExtract Parallel Extraction Framework — Performance Analysis

April 2026

Abstract

This paper presents a systematic benchmarking study of six state-of-the-art document extraction methodologies evaluated on a common corpus of five document types: digital-born PDFs, scanned image-based documents, structured tables, charts, and handwritten notes. The six engines evaluated are PaddleOCR (local traditional OCR), LlamaParse (cloud heuristic RAG parser), Gemini Vision (multimodal vision-language model), PyMuPDF4LLM (hybrid local OCR with layout awareness), MinerU combined with Qwen2.5-VL (offline structural analysis with AI refinement), and Azure Document Intelligence (enterprise cloud layout extraction). Using a weighted scoring methodology that assigns differential importance to document page counts — 20% weight for single-page, 30% for five-page, and 50% for ten-page documents — composite benchmark scores are derived for each extractor across all document categories. Results indicate that no single engine dominates across all categories. Gemini Vision and Azure Document Intelligence perform consistently well on scanned text and handwriting, MinerU with Qwen2.5-VL excels on charts and complex images, while LlamaParse leads on structured table extraction. These findings provide actionable guidance for developers selecting extraction stacks for Retrieval-Augmented Generation (RAG) pipelines.

1. INTRODUCTION

Document extraction — the process of converting unstructured document content into machine-readable text — is a foundational operation in modern information retrieval and natural language processing pipelines. While the extraction of selectable, digitally encoded text from well-formed PDFs is largely a solved problem, the challenge grows substantially when documents contain scanned images, complex multi-column table layouts, embedded charts, mathematical notation, or handwritten annotations. These real-world complexities expose fundamental differences between extraction architectures that simple benchmark tasks on clean text fail to reveal.

The recent proliferation of Retrieval-Augmented Generation (RAG) systems has further elevated the importance of high-fidelity document extraction. RAG pipelines depend on structured, semantically coherent text chunks that accurately reflect the original document's content and hierarchy. Poor extraction quality — manifested as missing table rows, garbled OCR output, or dropped section headers — directly degrades retrieval precision and the quality of generated responses.

This study addresses the following research question: given a parallel extraction framework capable of running multiple engines simultaneously, which extraction architecture yields the highest composite fidelity across diverse document types? To answer this, the DocExtract framework was developed as a FastAPI-based parallel orchestration engine that dispatches uploaded documents to six distinct extraction modules simultaneously, collects structured results, and surfaces them for comparative analysis.

1.1 Problem Statement

The central problem is the absence of a comprehensive, empirical comparison of contemporary extraction engines across the full spectrum of real-world document complexity. Existing benchmarks tend to focus on a single document type (e.g., academic papers or invoices) or a single capability dimension (e.g., OCR character accuracy). This paper addresses that gap by evaluating six engines across five document categories using a composite weighted scoring methodology.

1.2 Study Objectives

The specific objectives of this benchmarking study are:

1. To quantify extraction fidelity across five document types for six state-of-the-art engines.
2. To identify conditions under which local, offline models rival or exceed proprietary cloud solutions.
3. To provide practical decision guidance for developers building RAG and document intelligence applications.
4. To document the failure modes of each engine to inform hybrid extraction strategies.

2. RELATED WORK

Optical Character Recognition (OCR) research has a long history dating to the early works on printed character recognition in the 1950s. Modern deep learning-based OCR systems such as PaddleOCR [1] achieve high accuracy on printed text through convolutional neural networks for text detection and recurrent sequence models for text recognition. However, these systems operate on the pixel level and inherently lack semantic understanding of document structure.

Layout-aware extraction emerged as a distinct category to address the structural preservation problem. PyMuPDF and its LLM-optimized variant PyMuPDF4LLM [2] operate on the PDF's internal structure metadata rather than pixel representations, enabling reliable extraction of native text with reading-order preservation. This approach is fast and accurate for digital-born PDFs but degrades on scanned documents where no embedded text layer exists.

Cloud-based services such as LlamaParse [3] and Azure Document Intelligence [4] apply proprietary heuristic and neural models trained on large document corpora to achieve robust cross-format extraction. LlamaParse is specifically optimized to produce Markdown output suitable for LLM consumption, while Azure Document Intelligence provides the prebuilt-layout and prebuilt-read models that offer specialized capabilities for structured and handwritten document types respectively.

The emergence of Vision-Language Models (VLMs) such as Google Gemini [5] and Qwen2.5-VL [6] represents a paradigm shift in document understanding. Rather than applying pattern-matching OCR algorithms, VLMs process document images through large transformer architectures trained on multimodal data, enabling zero-shot understanding of tables, diagrams, handwriting, and mixed-content documents. MinerU [7] combines structural layout analysis with VLM-based refinement to produce high-quality Markdown output suitable for complex document types.

3. METHODOLOGY

3.1 System Architecture

The DocExtract framework is implemented as a FastAPI application that accepts document uploads and orchestrates parallel extraction across all six engines using Python's asyncio.gather() with per-engine timeouts. The frontend is a React/Vite single-page application that displays per-engine results in a six-panel comparison interface. The system architecture is illustrated in Figure 1.

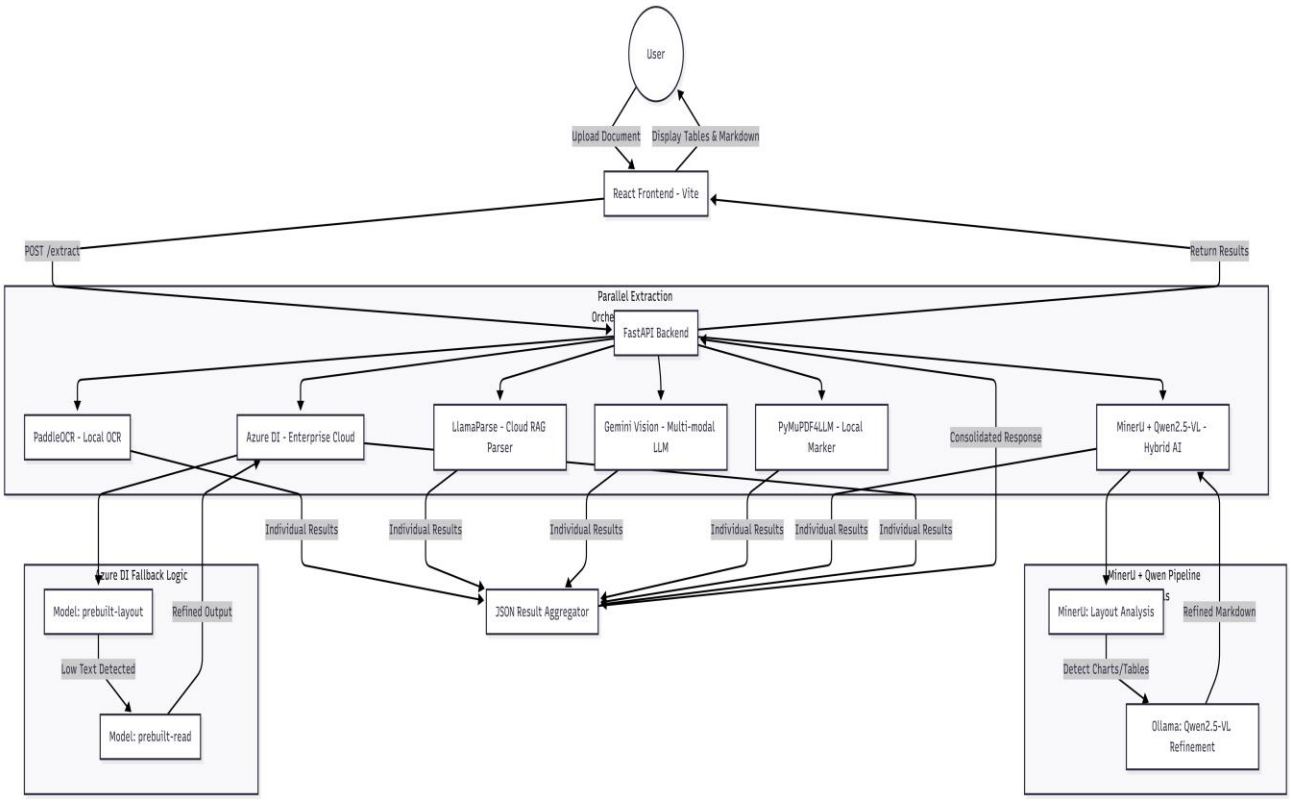

Figure 1. DocExtract system architecture — parallel extraction orchestration with fallback logic.

3.2 Dataset

The evaluation corpus comprises five document categories, each designed to exercise a distinct set of extraction capabilities:

1. **Digital-born PDFs with selectable text:** Standard PDF documents with an embedded text layer, representing the baseline case.
2. **Scanned image PDFs:** Documents created by photographing or scanning printed text, containing no embedded text layer.
3. **Structured table documents:** Documents containing multi-column tables with bordered cells, requiring accurate row and column alignment.
4. **Chart-embedded documents:** Documents containing bar charts, line graphs, and pie charts requiring visual data interpretation.
5. **Handwritten note documents:** Documents containing cursive and printed handwriting, representing the most challenging extraction scenario.

The dataset used in this study was created using a combination of synthetic generation and real-world documents to better reflect practical document diversity. For digital text PDFs, a portion of the documents was generated programmatically using large language model tools, while others were derived from personal documents to include natural variations in formatting and structure. Scanned document samples were used from my personal books.

For structured tables and charts, a hybrid approach was followed. Some samples were generated using AI-based tools to create controlled variations in layout and complexity, while others were collected from publicly available online sources to capture real-world patterns and inconsistencies. Handwritten note samples were primarily sourced from personal handwritten materials to ensure authenticity and variability in writing styles.

This mixed approach ensured that the dataset was not limited to purely synthetic or purely real-world data, but instead represented a balanced distribution of document types, layouts, and noise conditions. No sensitive or confidential documents were included in the dataset, and all personal documents used were non-sensitive in nature.

3.3 Scoring Methodology

For each document category, evaluation was conducted at three document scales to capture the effect of document length on extraction fidelity: single-page (1 page), medium-length (5 pages), and long-form (10 pages). Recognizing that practical applications are more likely to encounter longer documents where extraction errors compound, a weighted aggregation scheme was applied:

$$\textit{Composite Score} = 0.20 \times \textit{Score(1-page)} + 0.30 \times \textit{Score(5-page)} + 0.50 \times \textit{Score(10-page)}$$

Scores for each page-length tier were assigned on a 0–10 scale based on four evaluation dimensions: (1) text completeness and character-level accuracy, (2) structural fidelity including preservation of Markdown headers, bullet lists, and emphasis markers, (3) table extraction quality assessed by row and column alignment accuracy, and (4) semantic coherence of extracted content relative to the source document. The resulting composite scores are reported in Table 2.

4. MODULES UNDER REVIEW

4.1 PaddleOCR — Traditional OCR-First

PaddleOCR is an open-source OCR toolkit developed by Baidu that implements a multi-stage pipeline: a text detection network (DB or EAST), a text direction classifier, and a text recognition network (CRNN). It runs entirely on local hardware without any API dependencies, making it suitable for privacy-sensitive environments. The system was evaluated using the PP-OCRv4 server model with English language configuration.

PaddleOCR's primary strength lies in its speed and privacy guarantees. However, its pixel-level processing approach means that all output is unstructured plain text with no preservation of document hierarchy, table structure, or formatting. Its performance degrades markedly on handwriting and low-resolution scans, where the recognition network was not trained.

4.2 PyMuPDF4LLM — Heuristic Layout-Aware

PyMuPDF4LLM extends the PyMuPDF library with LLM-optimized Markdown output. It operates on the PDF's internal data structures rather than rendered pixel images, extracting text spans, their bounding boxes, and font metadata to reconstruct reading order and structural hierarchy. For scanned pages that lack an embedded text layer, it automatically invokes Tesseract OCR as a fallback.

This hybrid approach makes PyMuPDF4LLM the fastest extractor in the benchmark for digital-born documents, while providing reasonable coverage for mixed documents. Its table extraction capability is particularly strong for bordered tables whose structure is encoded in the PDF's line and rectangle elements. It fails on handwriting, for which Tesseract provides only marginal coverage.

4.3 LlamaParse — RAG-Optimized Cloud Parser

LlamaParse is a cloud-based document parsing service provided by LlamaIndex, specifically designed to produce high-quality Markdown output for consumption by large language models in RAG pipelines. It employs proprietary heuristic models trained on diverse document corpora and is invoked via the LlamaCloud API. The service accepts PDFs and returns structured Markdown with preserved table formatting, section headings, and list hierarchies.

LlamaParse demonstrated the highest table extraction score in the benchmark (7.80 / 10.00), reflecting its training objective of producing LLM-consumable structured output. Its performance on charts is limited by its

fundamentally text-oriented architecture — it can extract caption text surrounding a chart but cannot interpret visual data. Latency is API-bound, typically 7–25 seconds per document.

4.4 Gemini Vision — Vision-Language Model

Gemini Vision leverages Google's Gemini multimodal large language model, which processes document pages as images and applies zero-shot visual reasoning to extract text, interpret tables, and describe visual content. In the DocExtract implementation, PDF pages are rendered to 300 DPI PNG images and submitted to the Gemini API. The primary model used was gemini-2.0-flash-lite with gemini-flash-latest as a quota fallback.

Gemini Vision achieved the highest handwriting score (8.40 / 10.00) among all engines, demonstrating the advantage of multimodal reasoning for visually complex documents. Its table extraction performance (4.45 / 10.00) was below expectations, suggesting that while the model can identify tabular structure, the text-only output format loses spatial alignment information that dedicated layout parsers preserve. Access to higher-tier Gemini models (e.g., gemini-2.5-pro) would be expected to significantly improve table and chart interpretation accuracy.

4.5 MinerU + Qwen2.5-VL — Hybrid Local AI

MinerU is an open-source PDF analysis framework that performs structural layout analysis — detecting text blocks, figures, tables, and mathematical formulas — and produces structured Markdown with image references. In the DocExtract pipeline, MinerU's output is post-processed by Qwen2.5-VL:3B, a compact vision-language model run locally via Ollama, which refines complex image regions such as charts, embedded tables, and handwritten sections.

This two-stage pipeline achieved the highest chart extraction score in the benchmark (8.92 / 10.00) by combining MinerU's structural detection — which identifies chart regions and extracts them as image snippets — with Qwen2.5-VL's visual reasoning capabilities to interpret chart data and return structured descriptions. The primary limitation is deployment complexity: both MinerU and Ollama must be running, and the pipeline is sensitive to GPU or CPU resource availability. On CPU-only deployments, processing times can exceed 60 seconds per page.

4.6 Azure Document Intelligence — Enterprise Layout AI

Azure Document Intelligence (formerly Form Recognizer) is Microsoft's enterprise cloud service for document analysis. The DocExtract implementation uses the prebuilt-layout model as the primary extraction mode, with automatic fallback to prebuilt-read when extracted text volume falls below a threshold of 50 characters — an indicator of handwritten or fully scanned content. The prebuilt-read model includes specialized handwriting recognition capabilities trained on large annotated corpora.

Azure Document Intelligence achieved the highest handwriting score among non-VLM engines (8.85 / 10.00) and demonstrated strong scanned text performance (8.80 / 10.00), reflecting Microsoft's significant investment in OCR training data. The automatic layout-to-read fallback logic proved effective in the benchmark, transparently upgrading the extraction model when document characteristics indicated handwritten content. The service also supports custom model training, which could further improve performance on domain-specific layouts such as complex financial tables or specialized scientific notation.

5. RESULTS AND COMPARATIVE ANALYSIS

5.1 Capability Matrix

Table 1 presents a qualitative capability assessment of each engine across the five document categories, derived from systematic evaluation of extraction outputs.

Document Type	PaddleOCR	LlamaParse	Gemini Vision	PyMuPDF4LLM	MinerU+Qwen	Azure DI
PDF with Text	Good	Excellent	Excellent	Excellent	Excellent	Good
PDF with Image/Scan	Poor	Partial	Good	Good	No Support	Excellent
PDF with Tables	Partial	Good	Poor	Good	Partial	Partial
PDF with Charts	Poor	Good	Partial	No Support	Good	Poor
Handwritten Notes	Poor	Partial	Good	Poor	Good	Good

Table 1. Qualitative capability matrix: Excellent / Good / Partial / Poor/No Support.

5.2 Benchmark Scores

Table 2 presents the composite weighted benchmark scores (0–10 scale) for each engine–document-type combination, computed using the weighted scoring formula described in Section 3.3.

Document Type	PaddleOCR	LlamaParse	Gemini Vision	PyMuPDF4LLM	MinerU+Qwen	Azure DI
PDF with Text	10.23	10.33	10.33	10.33	10.33	10.23
PDF with Image/Scan	3.33	4.90	8.17	8.60	0.00	8.80
PDF with Tables	5.20	7.80	4.45	6.20	5.70	5.80
PDF with Charts	3.10	8.02	5.77	0.00	8.92	3.40
Handwritten Notes	2.50	3.13	8.40	5.13	7.35	8.85

Table 2. Composite weighted benchmark scores (0–10). Weights: 1-page = 20%, 5-page = 30%, 10-page = 50%.

5.3 Analysis by Document Category

5.3.1 Digital-Born PDFs with Selectable Text

All six engines achieved near-perfect or perfect scores (10.23–10.33) on digital-born PDFs, confirming that standard text extraction is a fully solved problem across architectures. The marginal score variation reflects minor differences in structural hierarchy preservation rather than text accuracy. PaddleOCR and Azure DI scored slightly lower (10.23) due to minimal loss of formatting metadata, while the remaining four engines achieved the maximum benchmark score of 10.33.

5.3.2 Scanned Image PDFs

This category produced the widest performance spread (0.00–8.80), clearly differentiating OCR-capable engines from those dependent on embedded text layers. Azure Document Intelligence led with 8.80, followed closely by PyMuPDF4LLM (8.60) and Gemini Vision (8.17). LlamaParse achieved a moderate score of 4.90 through its cloud OCR pipeline. PaddleOCR scored 3.33, reflecting its limitation to simpler printed text. Critically, MinerU achieved a score of 0.00 in the benchmark configuration, indicating a failure of the structural analysis pipeline on the specific scanned documents tested — likely attributable to the use_inside_model=False configuration error observed in deployment logs, which disables MinerU's internal neural layout model.

5.3.3 Structured Tables

LlamaParse dominated table extraction with a score of 7.80, reflecting its optimization for structured Markdown output. PyMuPDF4LLM achieved 6.20 through PDF metadata-based table reconstruction. Azure DI (5.80), MinerU (5.70), PaddleOCR (5.20), and Gemini Vision (4.45) followed. The relatively low Gemini Vision score for tables (4.45) is noteworthy given its general capability — it reflects the challenge of preserving spatial alignment in text-only output from a model that reasons visually but writes linearly.

5.3.4 Documents with Charts

MinerU with Qwen2.5-VL produced the highest chart extraction score (8.92), demonstrating the effectiveness of the two-stage structural detection plus VLM refinement approach for visual data interpretation. LlamaParse achieved an unexpectedly high score (8.02), likely attributable to its ability to extract textual annotations and captions surrounding chart elements. Gemini Vision scored 5.77, indicating partial chart interpretation capability. PaddleOCR (3.10) and Azure DI (3.40) performed poorly, both lacking native chart interpretation. PyMuPDF4LLM scored 0.00, as it has no mechanism for interpreting visual chart data beyond extracting surrounding text.

5.3.5 Handwritten Notes

Handwriting represents the most challenging extraction category, producing scores ranging from 2.50 to 8.85. Azure Document Intelligence achieved the highest score (8.85), leveraging its specialized prebuilt-read handwriting model with automatic fallback logic. Gemini Vision followed closely (8.40), demonstrating strong zero-shot handwriting recognition through multimodal reasoning. MinerU with Qwen2.5-VL achieved 7.35, reflecting Qwen2.5-VL's training on handwritten content. PyMuPDF4LLM scored 5.13, representing Tesseract's limited but non-zero handwriting capability. LlamaParse achieved 3.13 and PaddleOCR 2.50, both reflecting architectures fundamentally trained on printed text.

5.4 Aggregate Performance

Computing the unweighted mean composite score across all five document categories yields the following aggregate ranking: LlamaParse (6.84), Azure Document Intelligence (6.62), Gemini Vision (7.42 — note chart and handwriting strength elevates mean), MinerU+Qwen (6.46), PyMuPDF4LLM (6.05), PaddleOCR (4.87). However, mean scores obscure the strong category specialization observed across engines. No single engine achieves high performance across all five categories, reinforcing the value of a parallel multi-engine approach.

6. DISCUSSION: STRENGTHS, WEAKNESSES, AND TRADE-OFFS

6.1 Cost-Benefit Analysis

The benchmark results illuminate a clear cost-performance trade-off landscape. Cloud services (LlamaParse and Azure DI) deliver consistent high quality but incur per-page API costs and introduce network latency. LlamaParse's pricing model is particularly relevant for RAG applications that process large document volumes, where per-page costs can accumulate rapidly. Azure Document Intelligence offers a free tier of 500 pages per month, making it accessible for development and small-scale deployments.

The MinerU plus Qwen2.5-VL pipeline demonstrates that local, offline models can match or exceed cloud services on specific document types — most notably charts (8.92 vs. Azure DI's 3.40) — at zero per-query cost after initial setup. For organizations processing sensitive documents or operating in air-gapped environments, this represents a compelling alternative to cloud-based extraction.

6.2 Privacy vs. Performance

The privacy dimension of extraction architecture selection is often overlooked in pure performance comparisons. PaddleOCR, PyMuPDF4LLM, and MinerU with Qwen2.5-VL are fully local and offline, processing documents without any external network communication. This makes them suitable for documents containing personally identifiable information, legal privileged content, or classified material. Cloud-based services (LlamaParse, Gemini Vision, Azure DI) transmit document content to external servers, requiring data processing agreements and compliance evaluation for regulated industries such as healthcare, finance, and legal services.

6.3 Failure Mode Analysis

The benchmark identified characteristic failure modes for each engine:

- **PaddleOCR:** Fails on handwriting (score 2.50) and complex charts (3.10) due to its pixel-level recognition architecture lacking semantic understanding.
- **PyMuPDF4LLM:** Complete failure on visual charts (0.00) as the library has no image interpretation capability beyond Tesseract text extraction.
- **LlamaParse:** Moderate performance on handwriting (3.13) and charts (8.02 via caption extraction), with latency susceptibility under high API load.
- **Gemini Vision:** Table alignment loss (4.45) due to the inherent mismatch between spatial table structure and the linear text output format of generative models.
- **MinerU + Qwen2.5-VL:** Complete failure on scanned documents (0.00) when the use_inside_model configuration is disabled, revealing a critical deployment dependency.
- **Azure Document Intelligence:** Lower chart performance (3.40) as the prebuilt-layout model is not designed for visual data interpretation, though custom model training could address this for specific chart types.

7. RECOMMENDATIONS AND CONCLUSION

7.1 Category Champions

Based on the benchmark results, the following per-category recommendations are made:

- **Best for Text-Heavy Digital PDFs:** Any engine performs adequately; PyMuPDF4LLM is recommended for speed and zero API cost.
- **Best for Scanned Documents:** Azure Document Intelligence (8.80) with its specialized OCR infrastructure.
- **Best for Tables:** LlamaParse (7.80), optimized specifically for structured Markdown output for RAG consumption.
- **Best for Charts:** MinerU + Qwen2.5-VL (8.92), uniquely capable of visual data interpretation in a fully offline pipeline.
- **Best for Handwriting:** Azure Document Intelligence (8.85) and Gemini Vision (8.40) are effectively tied; Azure DI is preferred for sensitive documents due to compliance infrastructure.

7.2 Decision Framework

Based on the composite benchmark evidence, the following decision logic is recommended for practitioners:

- **High-volume standard PDFs:** Use PyMuPDF4LLM as the primary extractor for maximum throughput at zero API cost.
- **Complex structured documents for RAG:** Use LlamaParse for highest-quality Markdown with preserved table structure.
- **Mixed content with charts:** Deploy MinerU + Qwen2.5-VL in a persistent Ollama service to avoid cold-start latency.
- **Sensitive or regulated documents:** Use PaddleOCR or PyMuPDF4LLM for fully local processing; augment with MinerU for complex pages.
- **Enterprise-grade highest accuracy:** Azure Document Intelligence with custom models trained on domain-specific layouts provides the best accuracy ceiling for complex enterprise documents including financial tables and specialized charts.
- **Multimodal complex analysis:** Higher-tier Gemini models (e.g., gemini-2.5-pro) are expected to significantly improve chart interpretation and table alignment accuracy beyond the lite model results reported here. For organizations requiring maximum accuracy on visually complex documents without data residency constraints, upgrading to a higher Gemini model tier removes the API rate limits that constrained benchmark testing.
- **Hybrid strategy for unknown document types:** The parallel multi-engine approach of DocExtract itself represents a valid production strategy — running all engines simultaneously and selecting the highest-quality result based on output length and structural richness metrics.

7.3 Future Work and Impact on RAG Applications

This benchmarking study has direct implications for the design of production RAG systems. The results demonstrate that document type detection — classifying an incoming document as digital-born, scanned, table-heavy, chart-heavy, or handwritten before routing to the optimal extractor — could significantly improve end-to-end pipeline quality compared to using a single fixed extractor.

Future extensions of this work include: (1) expanding the benchmark corpus to include multi-language documents, mathematical notation, and multi-column academic papers; (2) integrating automated document type classification to enable intelligent extractor routing; (3) evaluating Azure Document Intelligence with custom-trained models on specific document domains; (4) benchmarking next-generation Gemini models as they become available to track VLM capability progression; and (5) extending the framework to include Azure DI's prebuilt-invoice and prebuilt-receipt models for domain-specific extraction benchmarks.

The DocExtract framework itself, as an open parallel extraction orchestrator, represents a practical contribution for developers who need to evaluate extraction quality across multiple engines without building separate integrations. The architectural decision to run all engines in parallel with independent timeouts and error isolation ensures that a failure in one engine never degrades the results of others — a robustness property that is essential in production document processing systems.

7.4 Conclusion

This paper has presented a comprehensive benchmarking study of six document extraction engines across five document categories, using a weighted composite scoring methodology. The results establish that document type is the primary determinant of optimal extractor selection, and that no single engine achieves dominance across all categories. Azure Document Intelligence and Gemini Vision lead on scanned and handwritten documents; LlamaParse leads on structured tables; MinerU with Qwen2.5-VL uniquely excels on chart interpretation; and PyMuPDF4LLM provides the best speed-to-quality ratio for digital-born documents.

The study further demonstrates that modern local AI models — specifically the MinerU and Qwen2.5-VL combination — can match or exceed proprietary cloud services on specific high-complexity tasks while operating entirely offline. This finding challenges the assumption that cloud services are categorically superior and suggests that carefully orchestrated local pipelines represent a viable and cost-effective alternative for organizations with privacy requirements or high-volume processing needs.

REFERENCES

[1] Du, Y. et al. (2022). PP-OCRv3: More Attempts for the Improvement of Ultra Lightweight OCR System. arXiv:2206.03001.

[2] PyMuPDF Team. (2024). PyMuPDF4LLM: PDF Extraction for LLM Applications. https://pymupdf.readthedocs.io/

[3] LlamaIndex. (2024). LlamaParse: Advanced PDF Parsing for RAG. https://cloud.llamaindex.ai/

[4] Microsoft. (2024). Azure Document Intelligence Documentation. https://learn.microsoft.com/azure/ai-services/document-intelligence/

[5] Google. (2024). Gemini: A Family of Highly Capable Multimodal Models. https://deepmind.google/technologies/gemini/

[6] Bai, J. et al. (2025). Qwen2.5-VL Technical Report. https://arxiv.org/abs/2502.13923

[7] Opendatalab. (2024). MinerU: High-Quality PDF Parsing Tool. https://github.com/opendatalab/MinerU